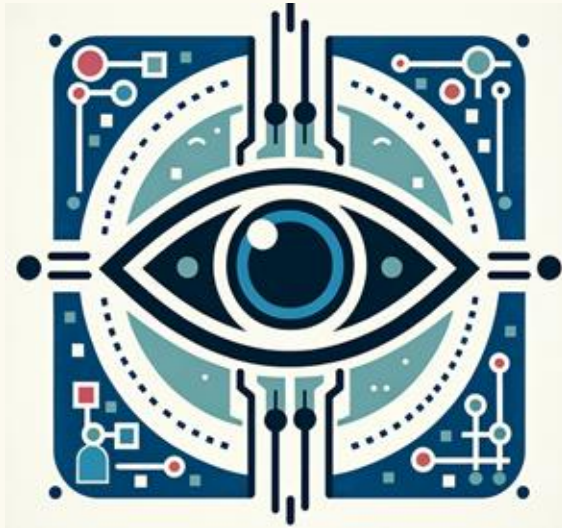
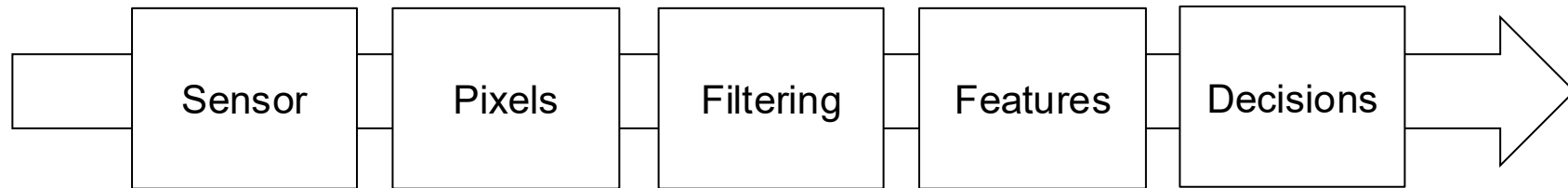


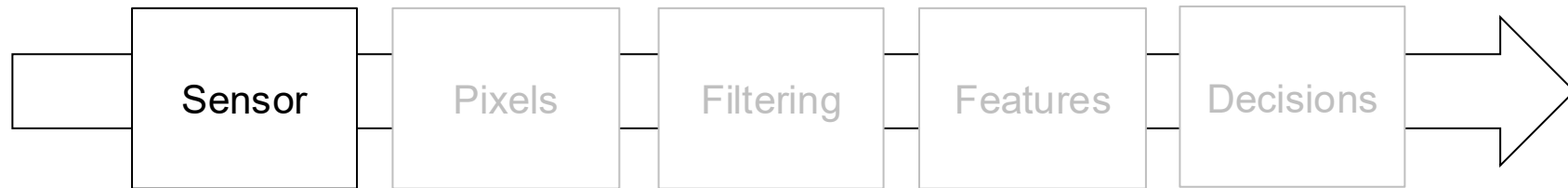


COMPUTER VISION

Image Basics



Traditional pipeline



What is an image?

Camera Obscura

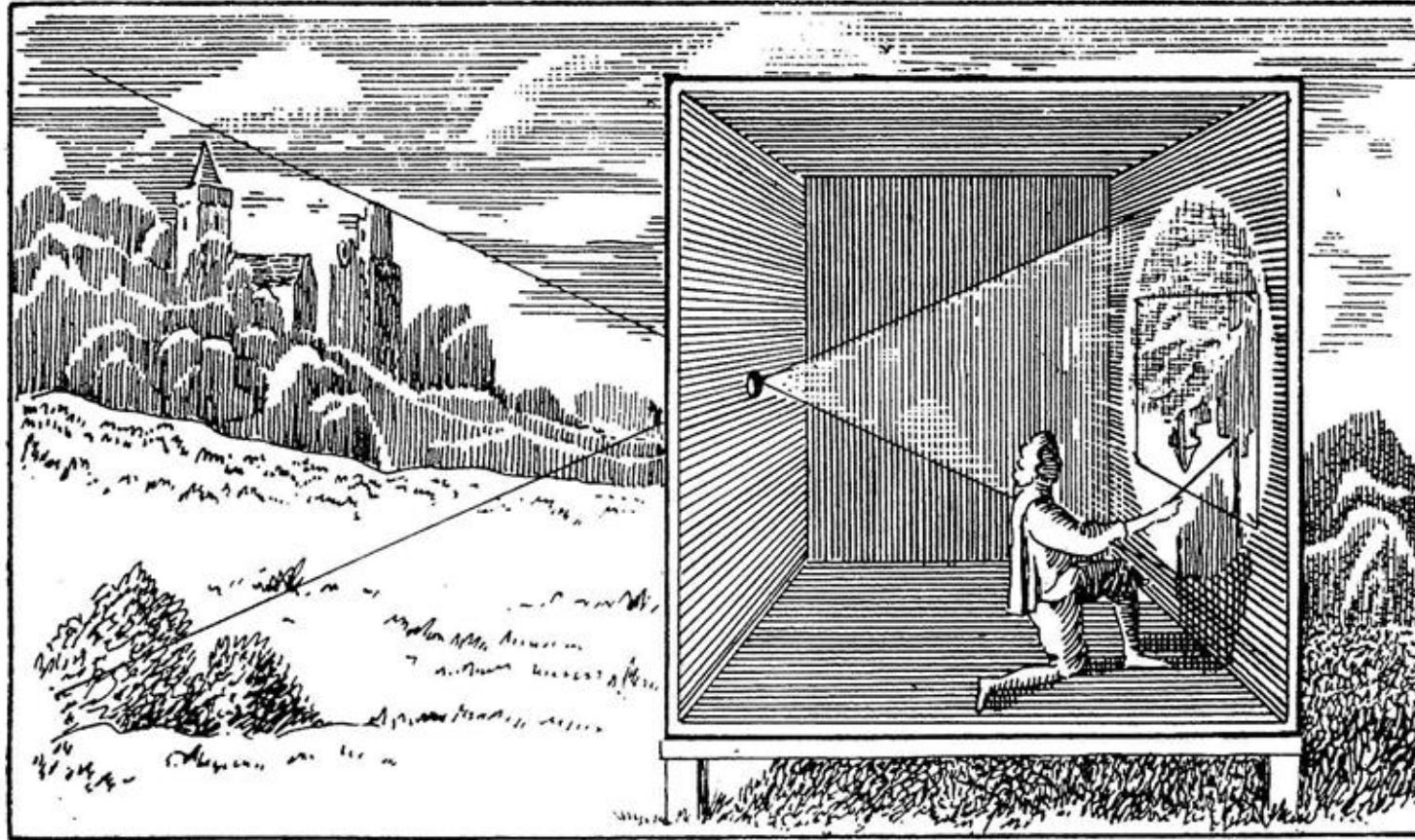
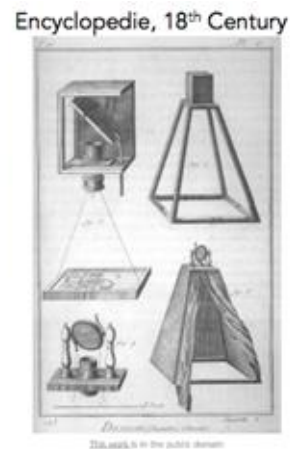
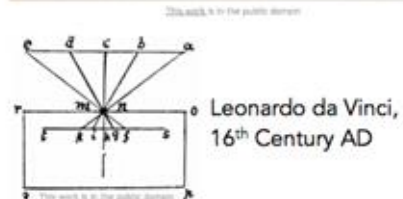
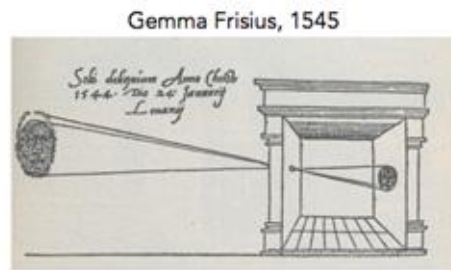


Figure from <https://1stpersontech.wordpress.com>

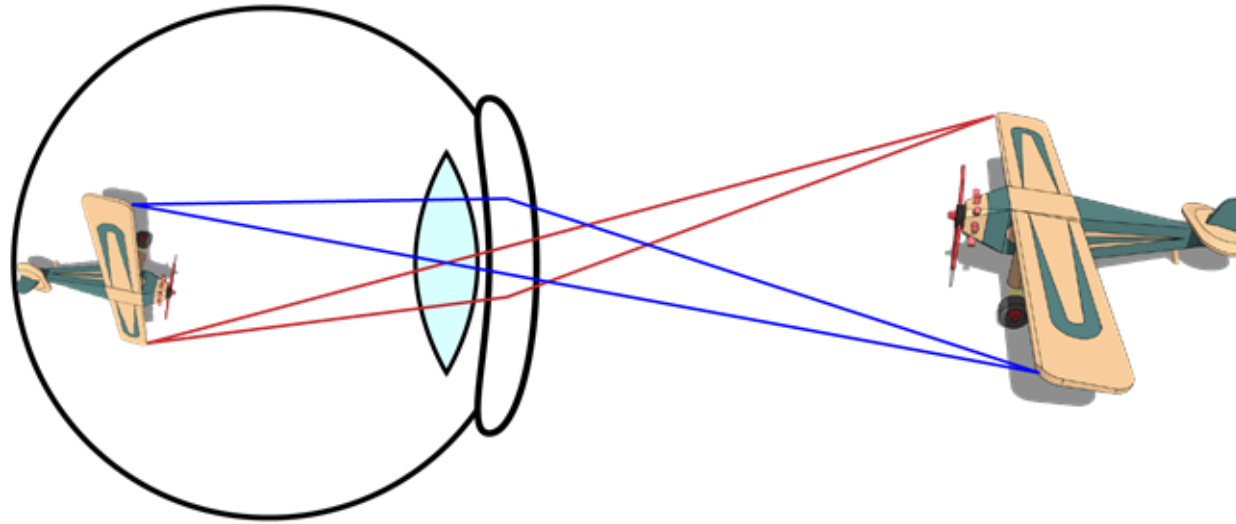
Projection of the real world

Cameras

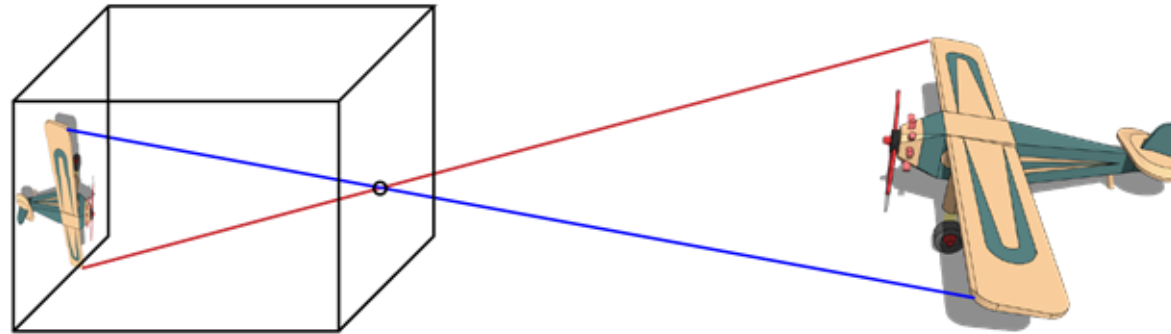
- Camera obscura and pinhole camera
 - Light from a scene passes through the aperture and projects an inverted image on the opposite side of the box
- Photography in the 19th century
- Now we have cameras everywhere



Eyes: projection onto retina



Model: pinhole camera



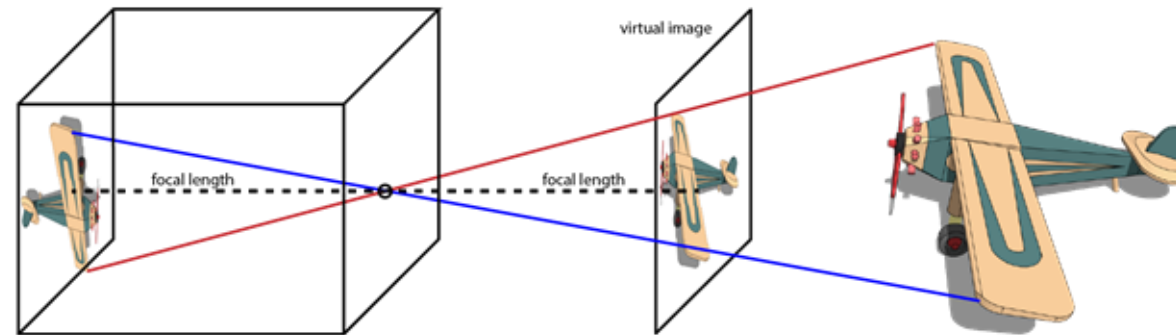
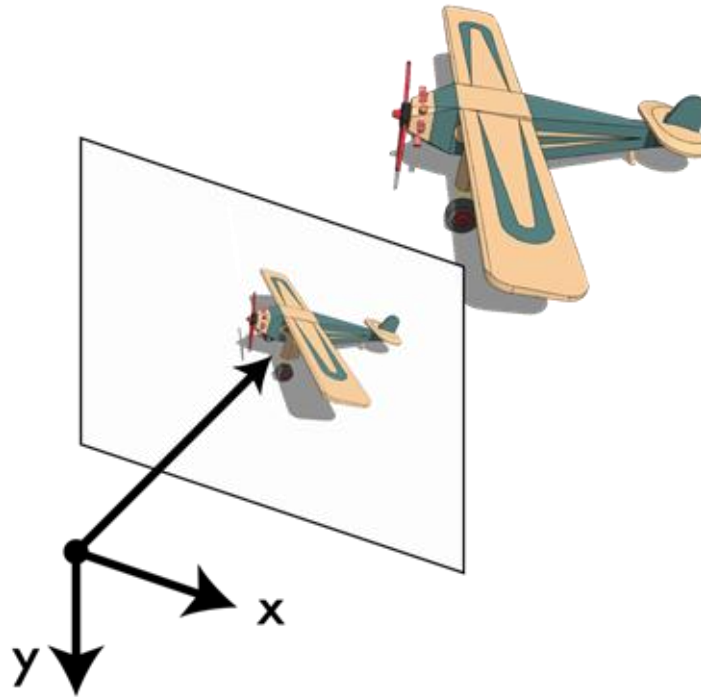
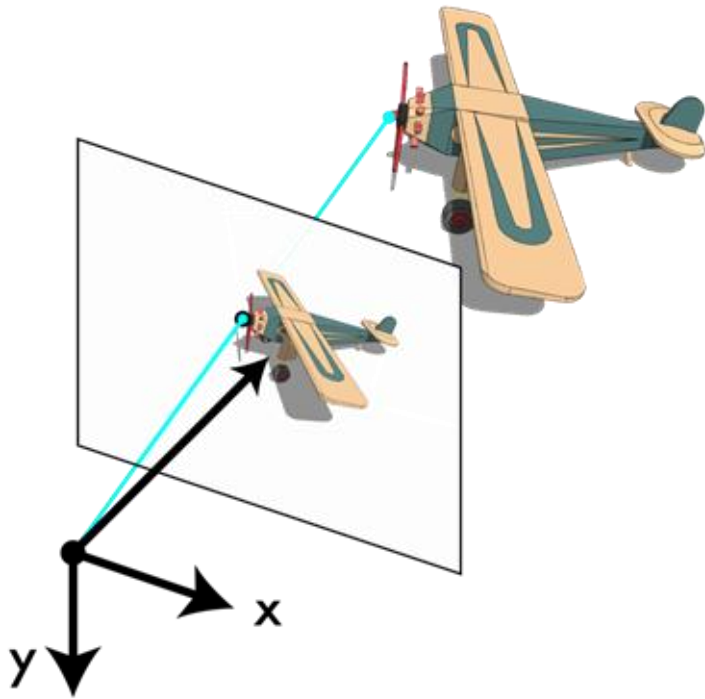
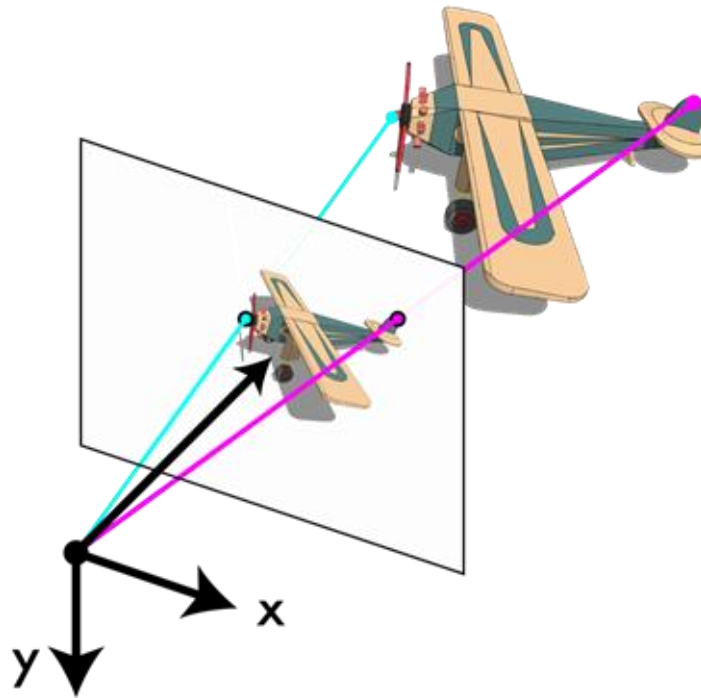
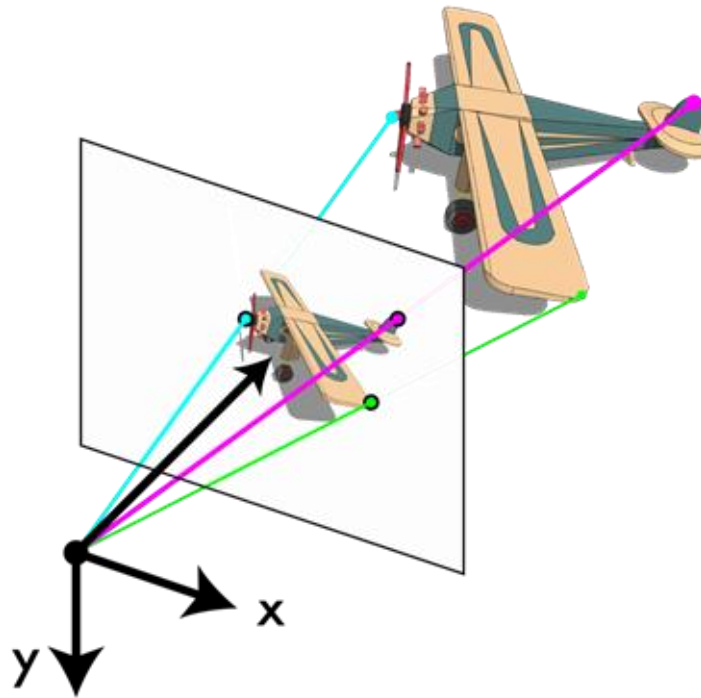


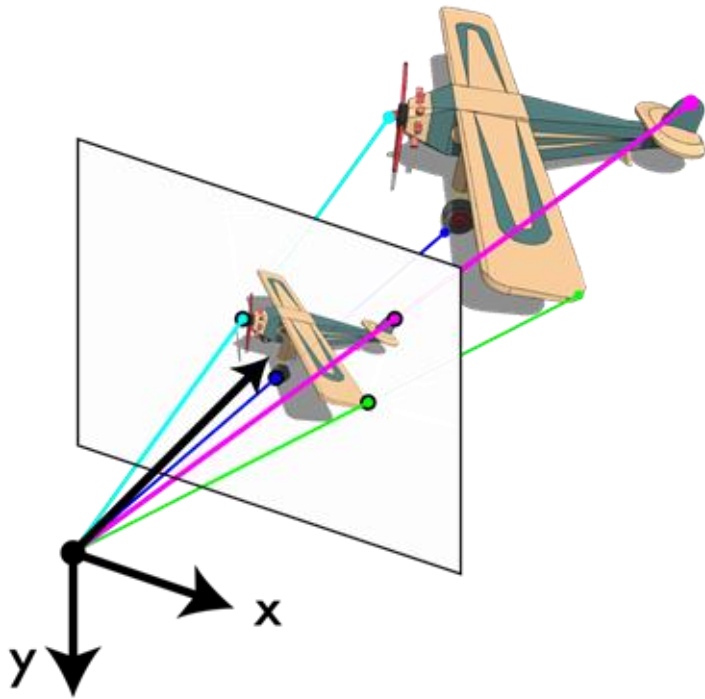
Image: 3D $\rightarrow$ 2D projection of the world



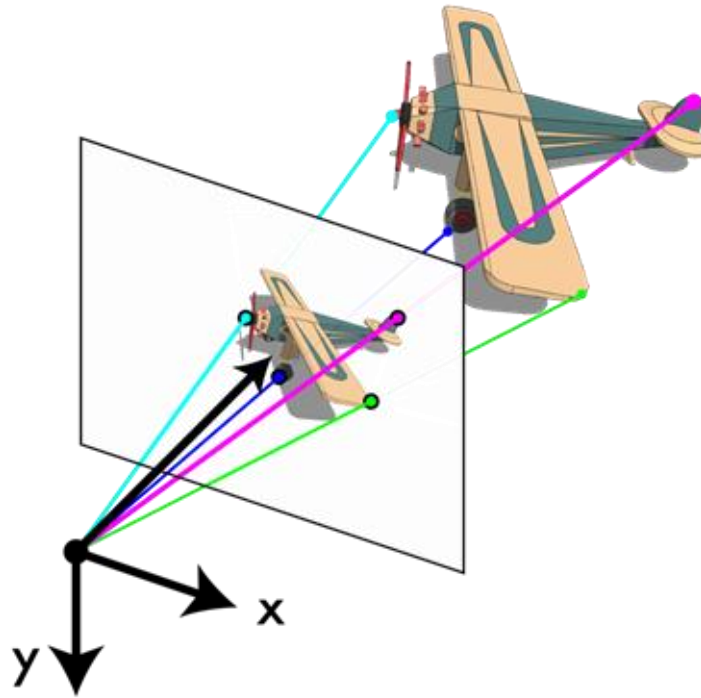




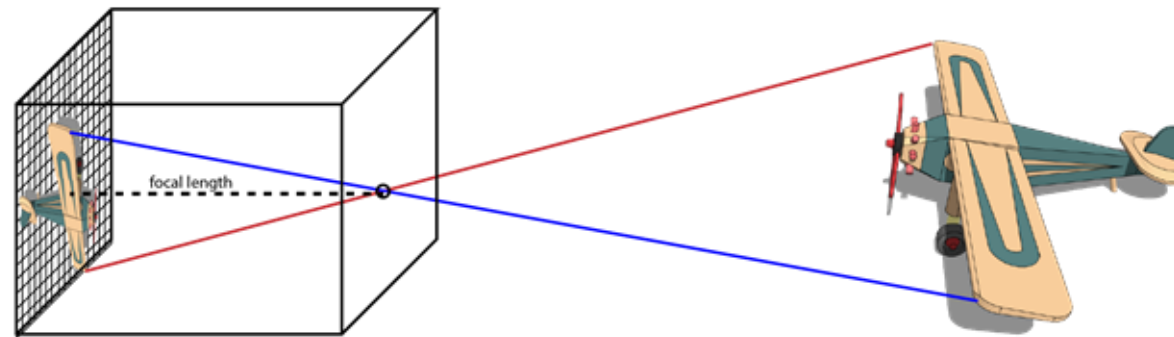




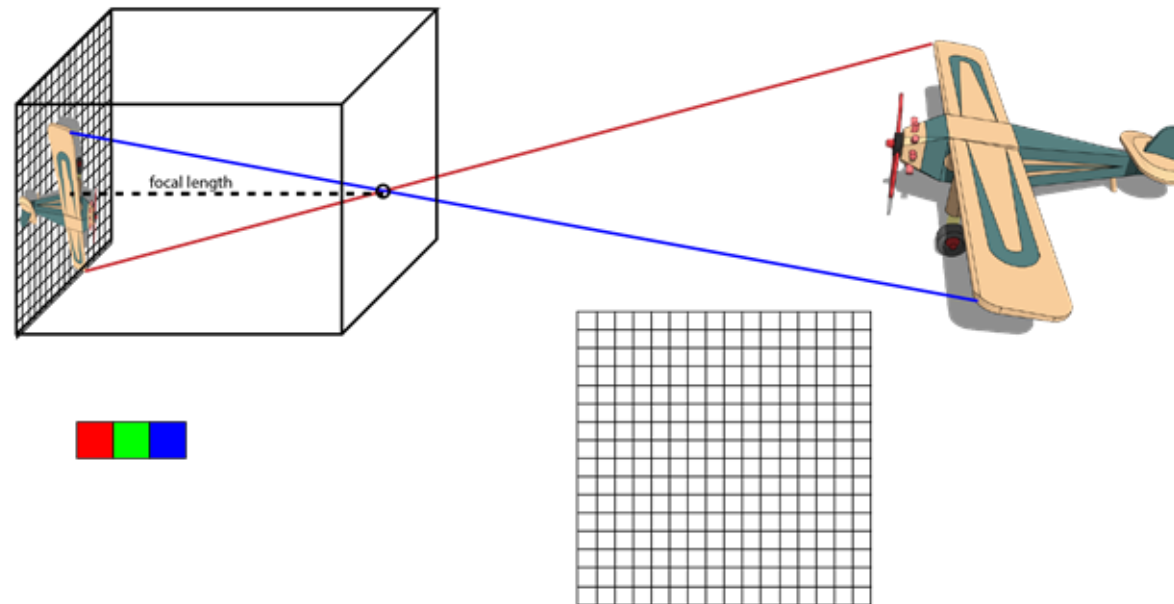
At each point we record incident light



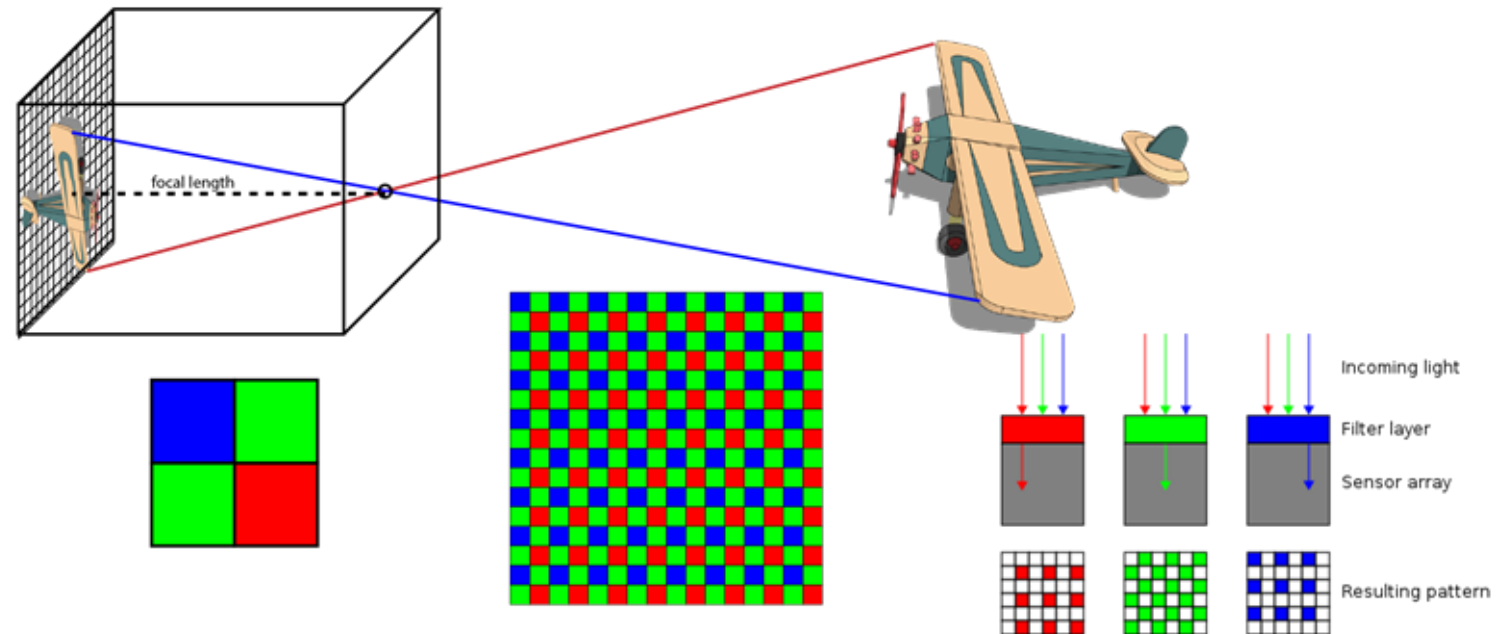
At each point we record incident light

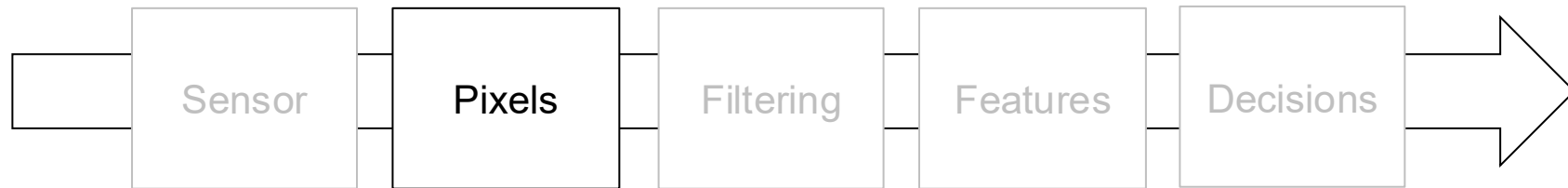


How do we record color?

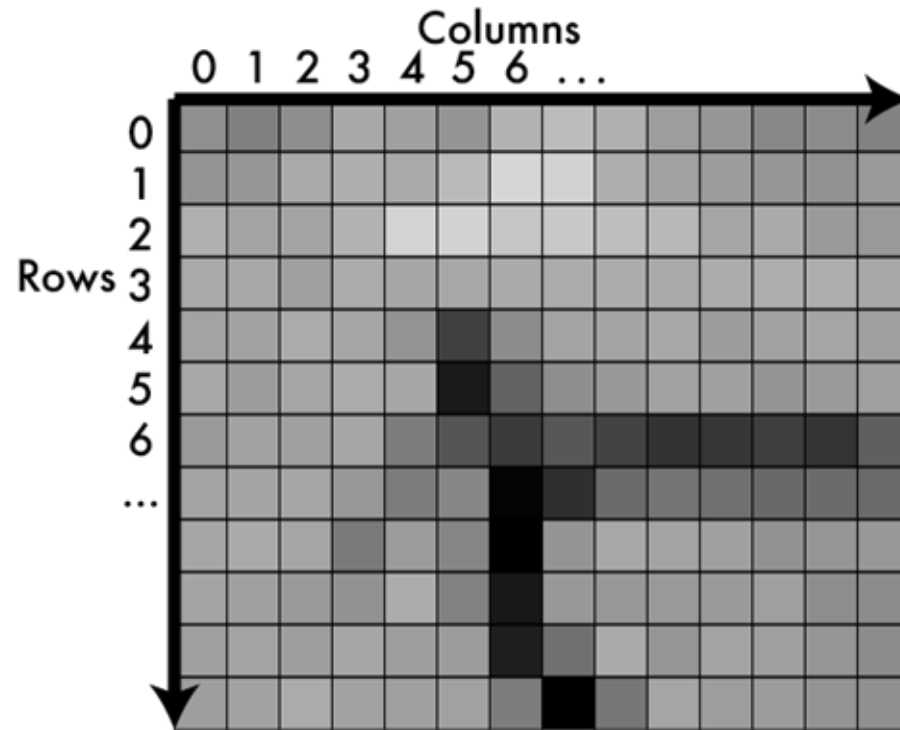


Bayer pattern for CMOS sensors





An image is a matrix of light



Values in matrix = how much light

		Columns													
		0	1	2	3	4	5	6	...						
Rows	0	100	102	107	102	132	146	136	156	148	122	115	104	105	103
	1	100	102	107	102	132	146	136	156	148	122	115	104	105	103
	2	100	102	107	102	132	146	136	156	148	122	115	104	105	103
	3	100	102	107	102	132	146	136	156	148	122	115	104	105	103
	4	100	102	107	102	132	146	136	156	148	122	115	104	105	103
	5	100	102	107	102	132	30	60	156	148	122	115	104	105	103
	6	100	102	107	102	132	40	20	50	32	20	20	24	30	62
	...	100	102	107	102	132	71		156	51	57	57	58	62	58
		100	102	107	102	132	69		156	148	122	115	104	105	103
		100	102	107	102	132	89	12	156	148	122	115	104	105	103
		100	102	107	102	132	146	13	45	148	122	115	104	105	103
		100	102	107	102	132	146	46		42	122	115	104	105	103

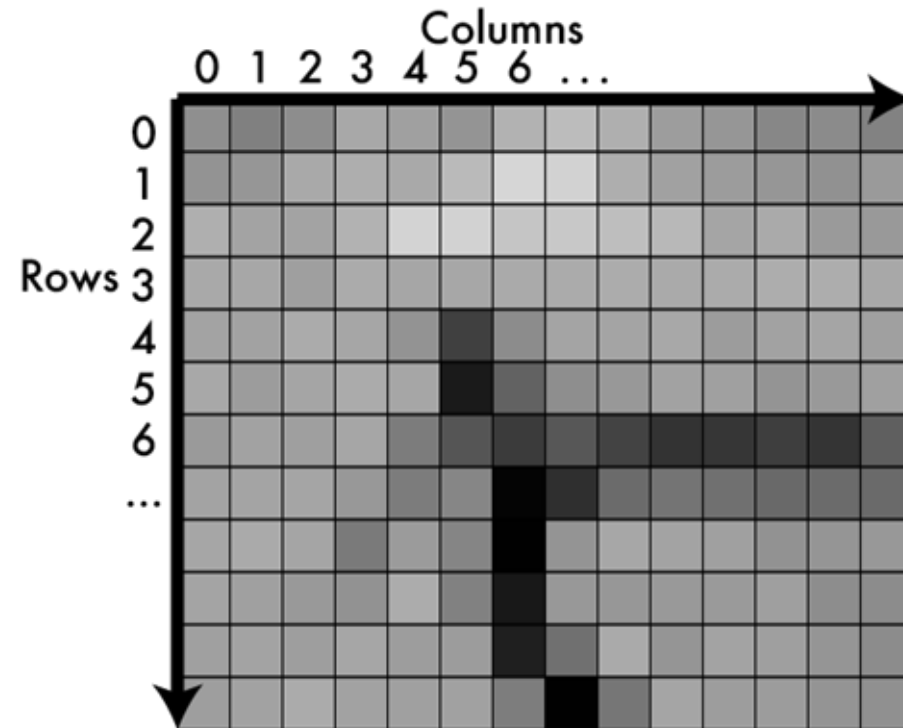
Values in matrix = how much light

- Higher = more light
- Lower = less light
- Bounded
 - No light = 0
 - Sensor/device limit = max
 - Typical ranges:
 - [0-255], fit into byte
 - [0-1], floating point
- Called pixels (*picture elements*)

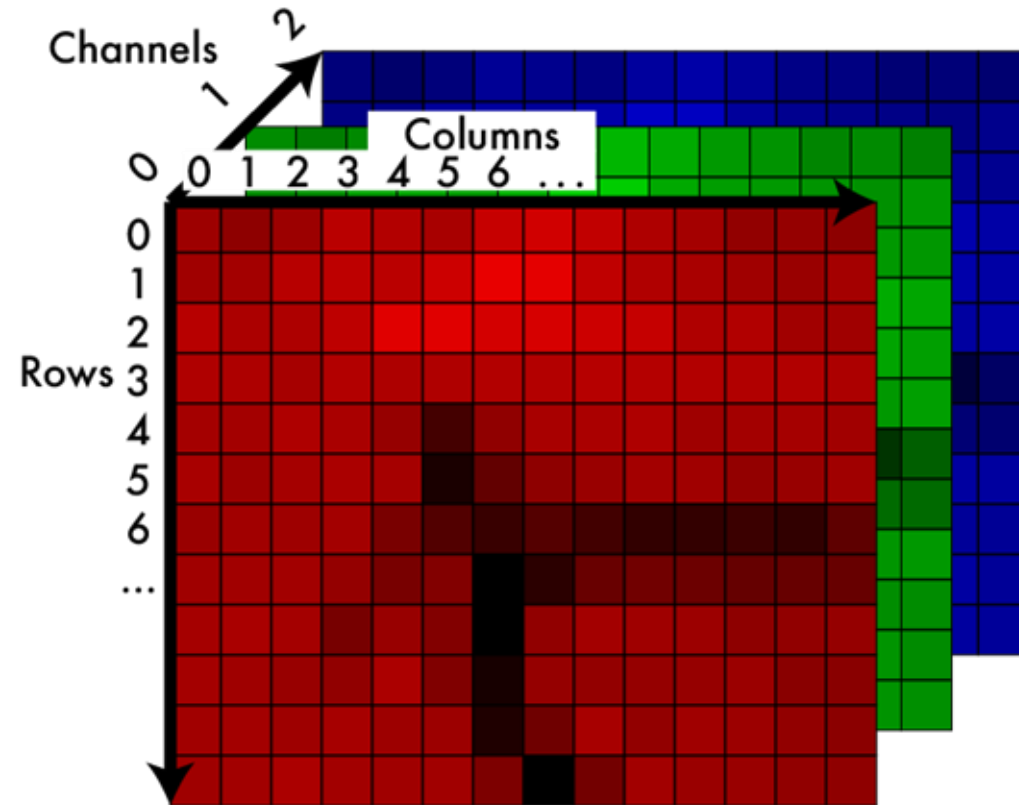
		Columns													
		0	1	2	3	4	5	6	...						
Rows	0	100	102	107	102	132	146	136	156	148	122	115	104	105	103
	1	100	102	107	102	132	146	136	156	148	122	115	104	105	103
	2	100	102	107	102	132	146	136	156	148	122	115	104	105	103
	3	100	102	107	102	132	146	136	156	148	122	115	104	105	103
	4	100	102	107	102	132	146	136	156	148	122	115	104	105	103
	5	100	102	107	102	132	30	60	156	148	122	115	104	105	103
	6	100	102	107	102	132	40	20	50	32	20	20	24	30	62
	...	100	102	107	102	132	71		156	51	57	57	58	62	58
		100	102	107	102	132	69		156	148	122	115	104	105	103
		100	102	107	102	132	89	12	156	148	122	115	104	105	103
	100	102	107	102	132	146	13	45	148	122	115	104	105	103	
	100	102	107	102	132	146	46		42	122	115	104	105	103	

Addressing pixels

- How to index images:
 - (x,y)
 - Like cartesian coordinates
 - $(3,6)$ is column 3 row 6
 - (r,c) or (h,w)
 - Like matrix notation
 - $(3,6)$ is row 3 column 6

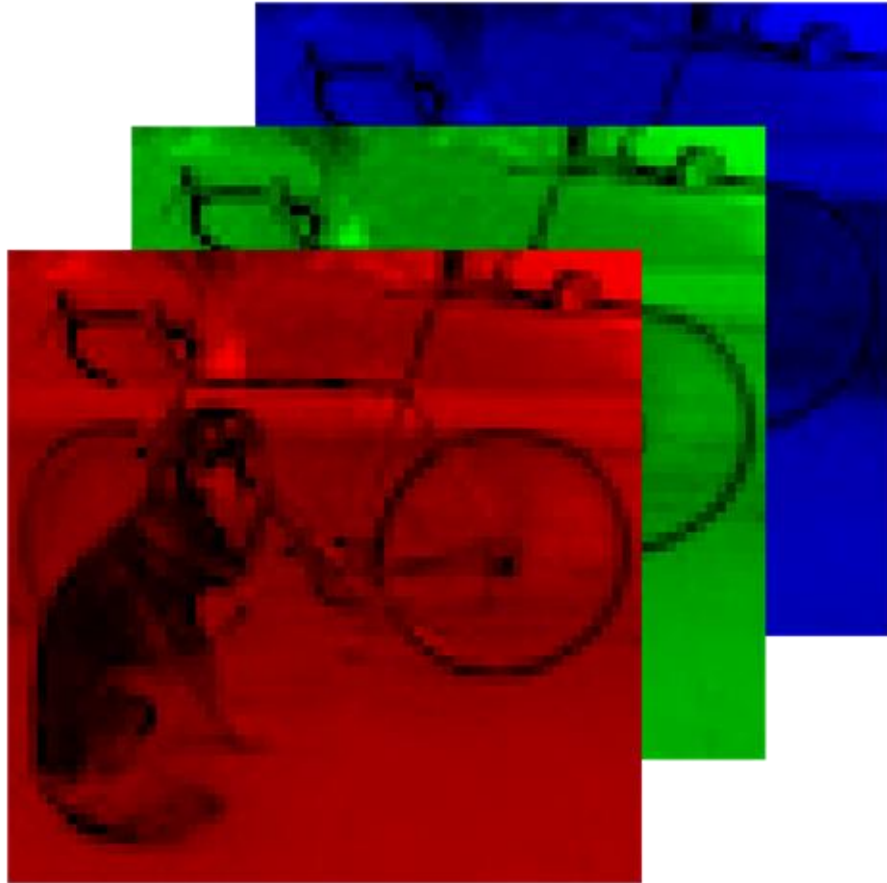


Color image: 3D tensor in a given colorspace



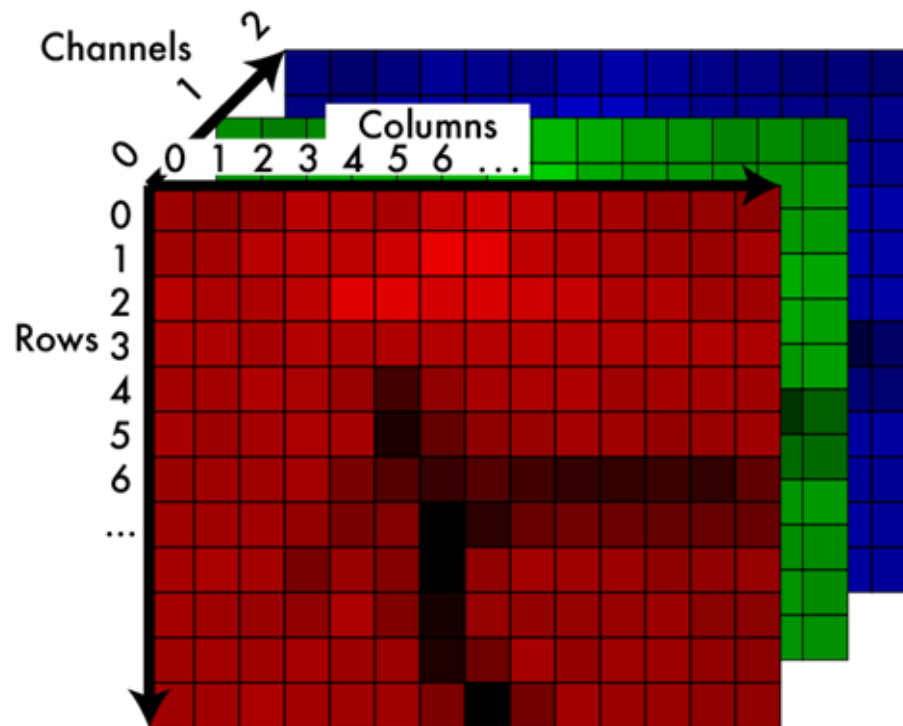
RGB information in separate “channels”

- We can match “real” colors using a mix of primaries.
- Each channel encodes one primary. Adding the light produced from each primary mimics the original color.

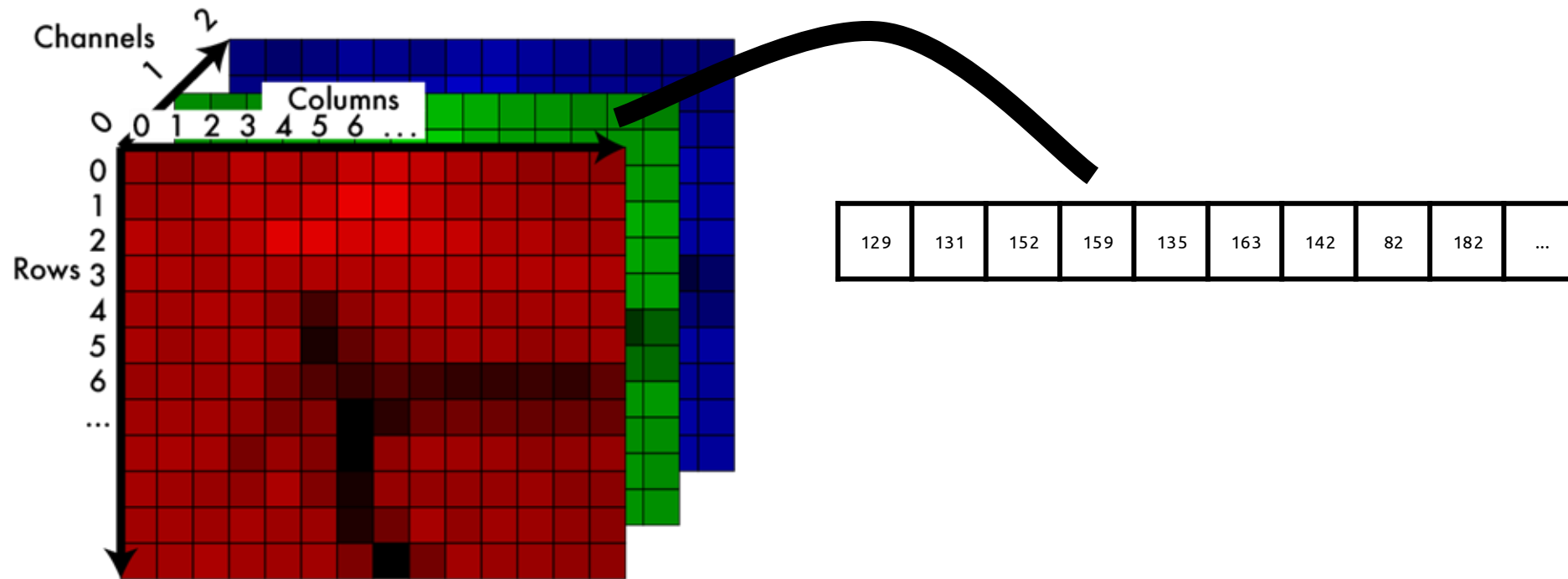


Addressing pixels

- (x, y, z)
 - $(1, 2, 0)$:
 - column 1, row 2, channel 0
- (c, h, w)
 - $(1, 2, 0)$:
 - channel 1, row 2, column 0
- Basically the same
→ consistency is the most important thing

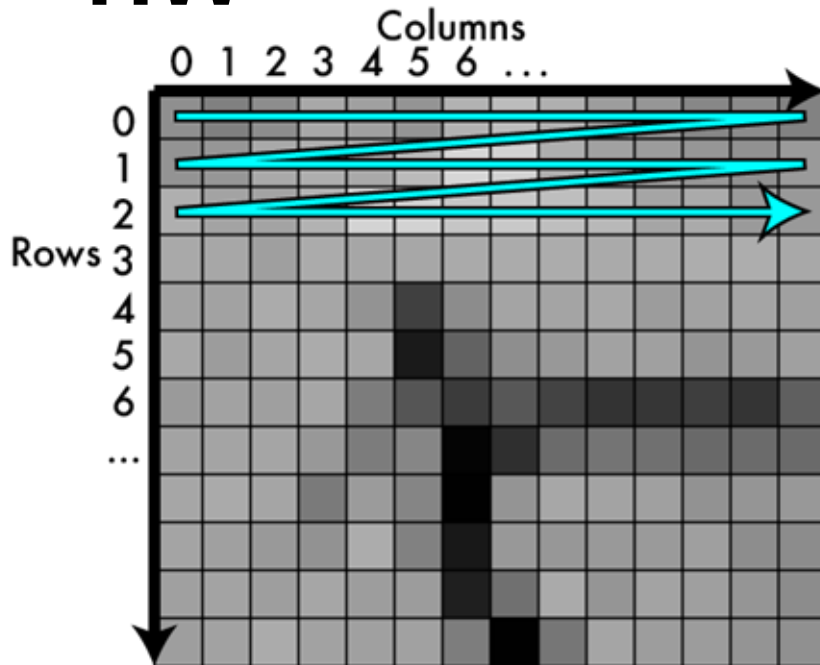


How do we store them?

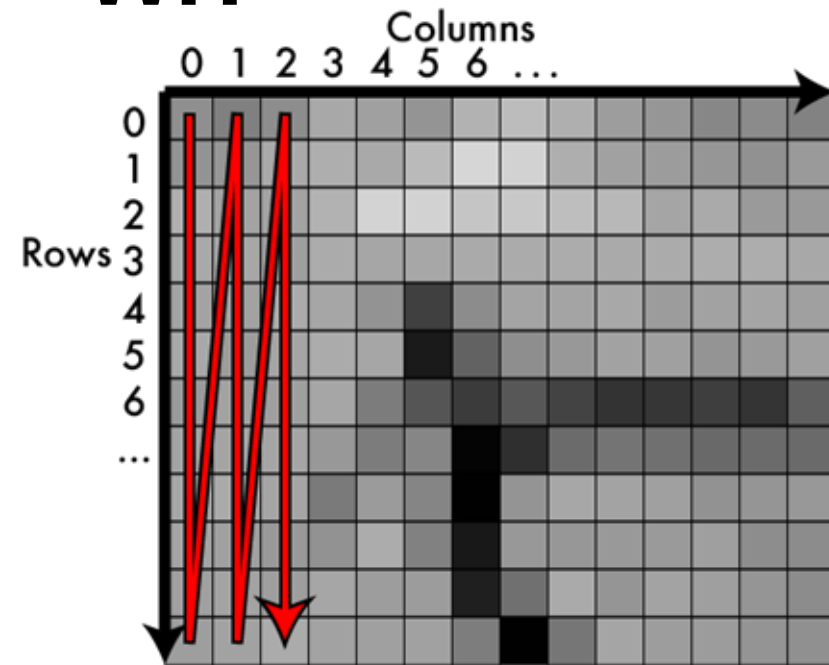


Storage: row major vs column major

HW

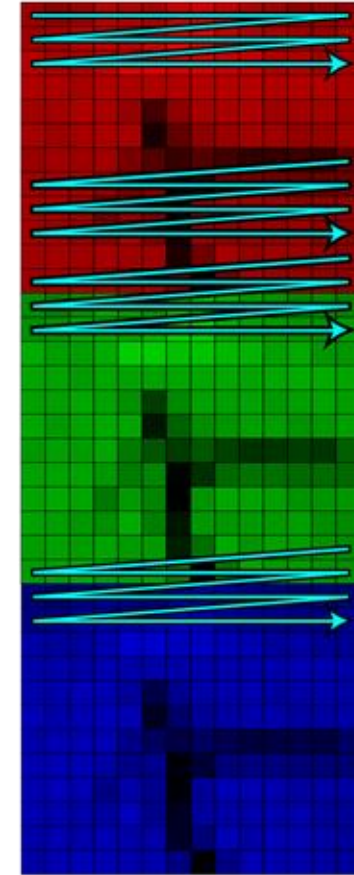
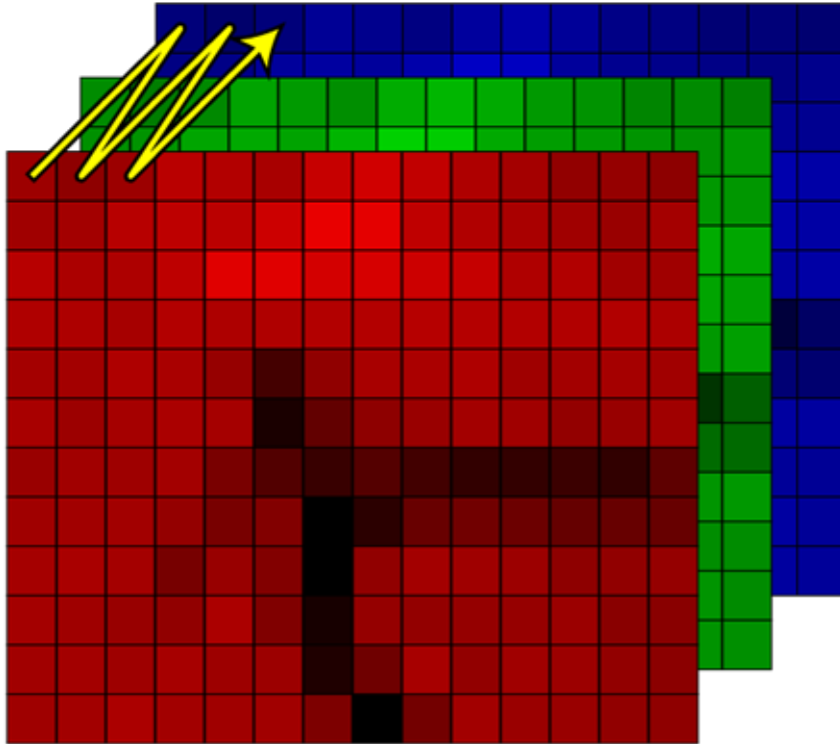


WH

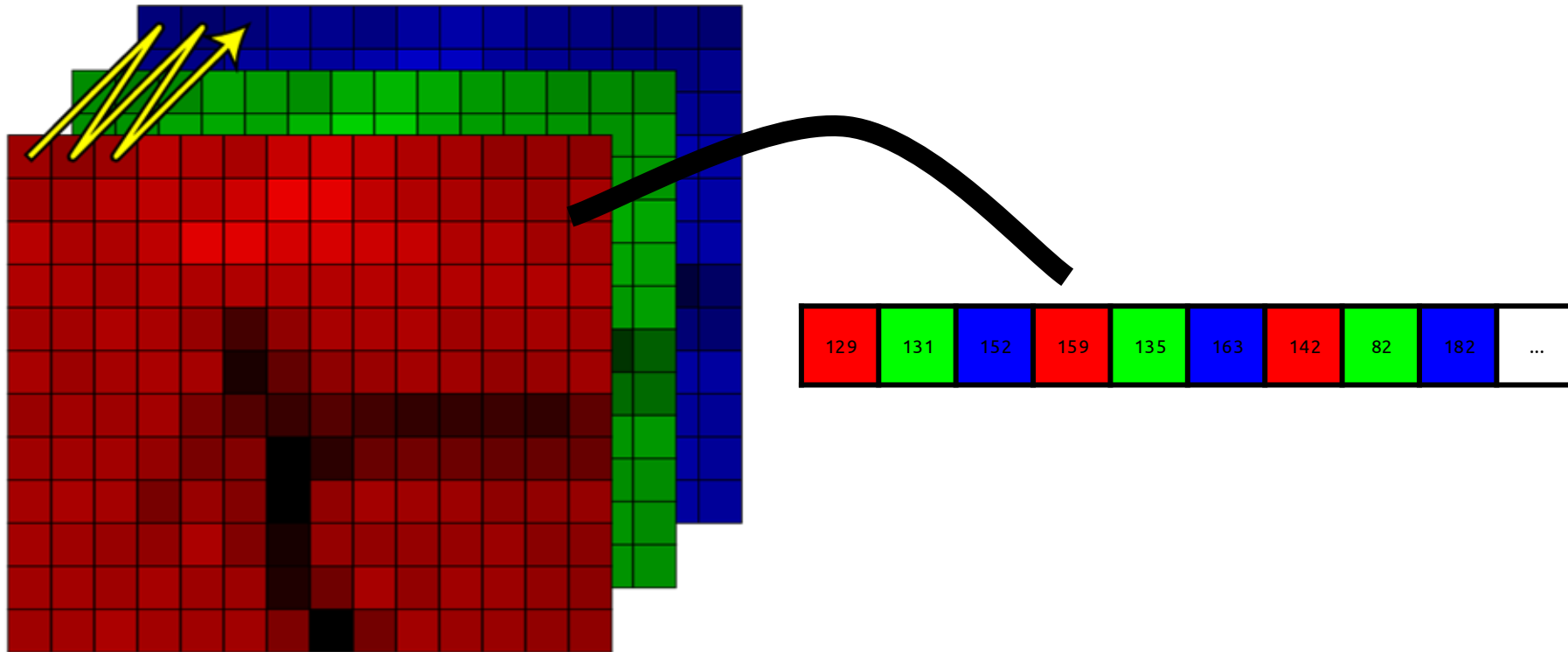


Typically, row-major or HW is used

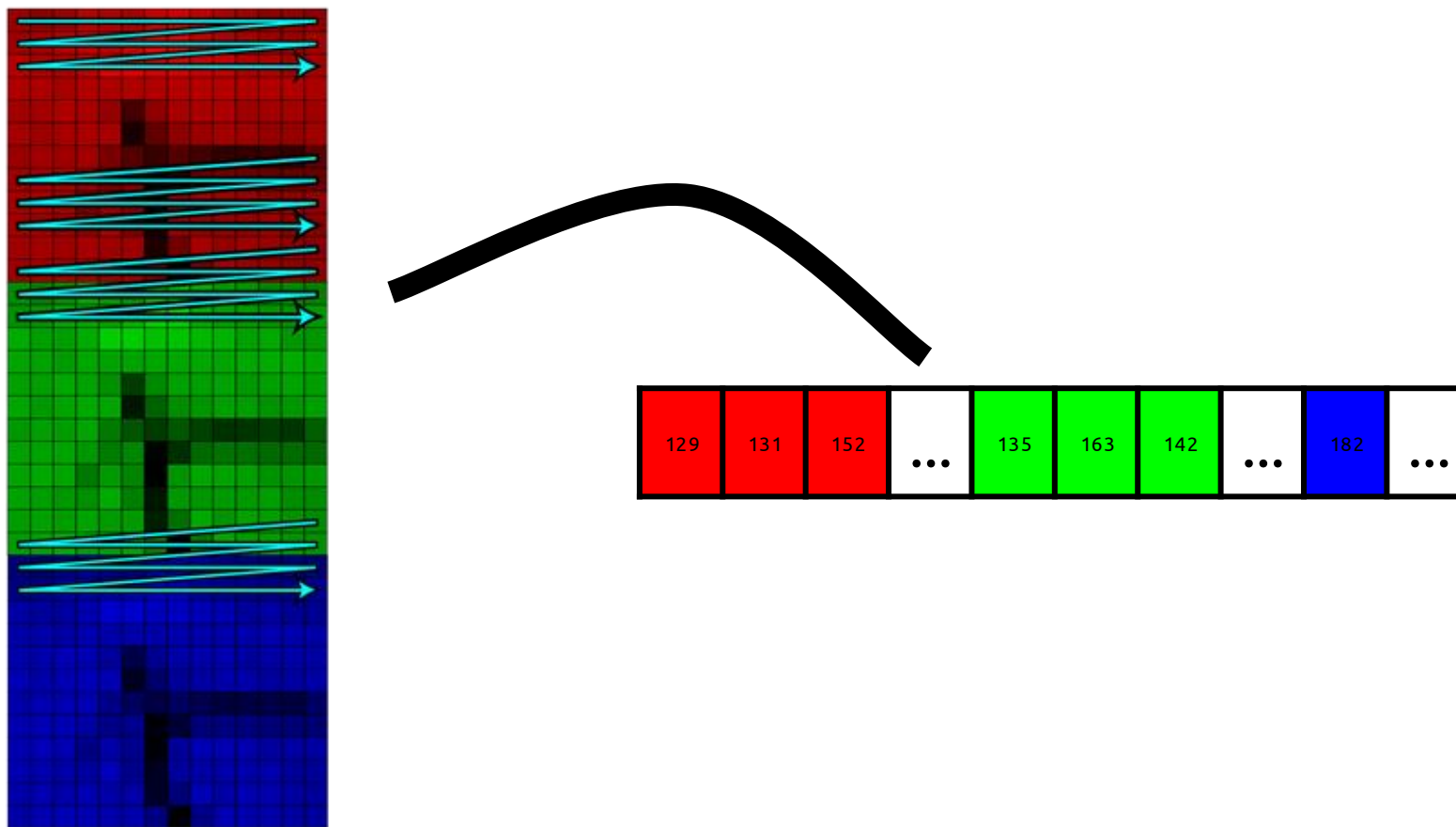
In 3D we have more choices!



HWC: channels interleaved



CHW: channels separated



Quick quiz

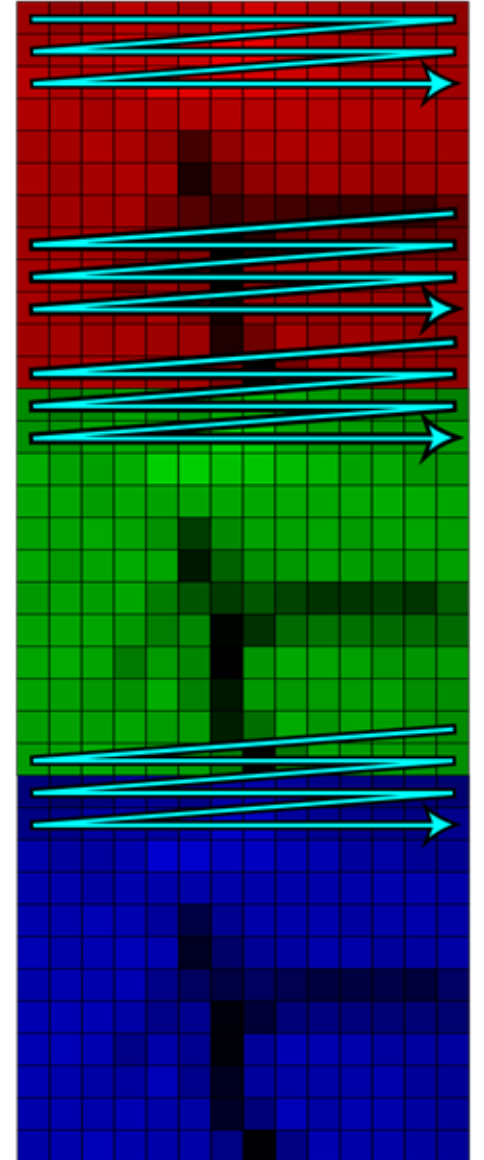
In an array for a 1920 x 1080 x 3 image what entry would contain the pixel (15,192,2)?

In general for (x,y,z) of image (W,H,C)

$$x + y*W + z*W*H$$

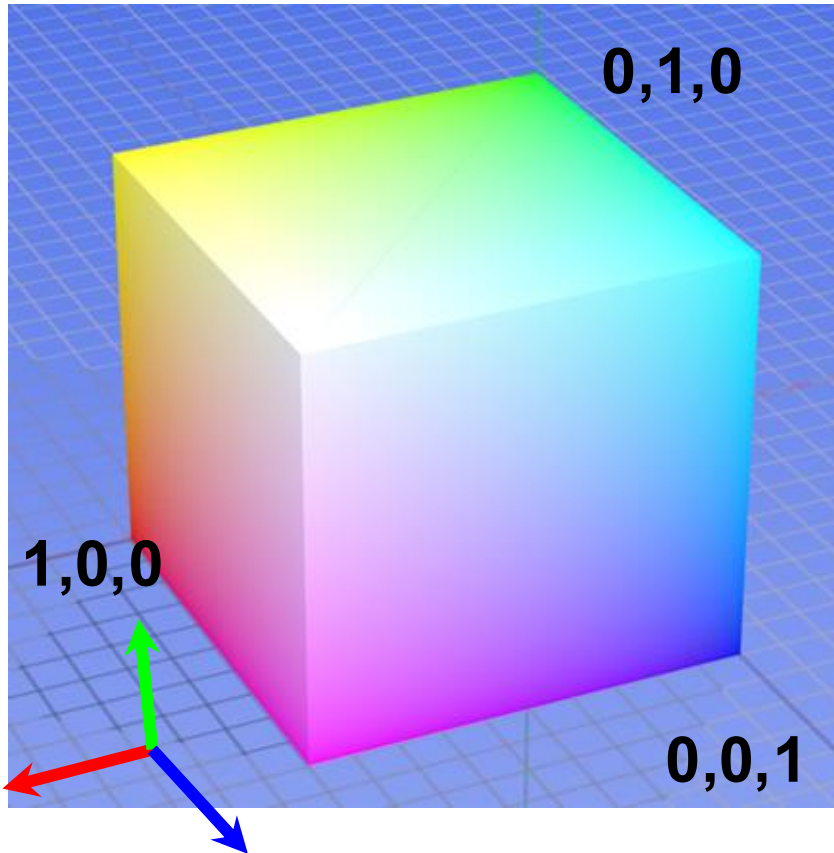
$$15 + 192*1920 + 2*1920*1080 = 4,515,855$$

Remember, *everything is 0 indexed*



Color spaces: RGB

Default color space



Any color = $r \cdot R + g \cdot G + b \cdot B$

- Strongly correlated channels
- Non-perceptual



R = 1
(G=0,B=0)

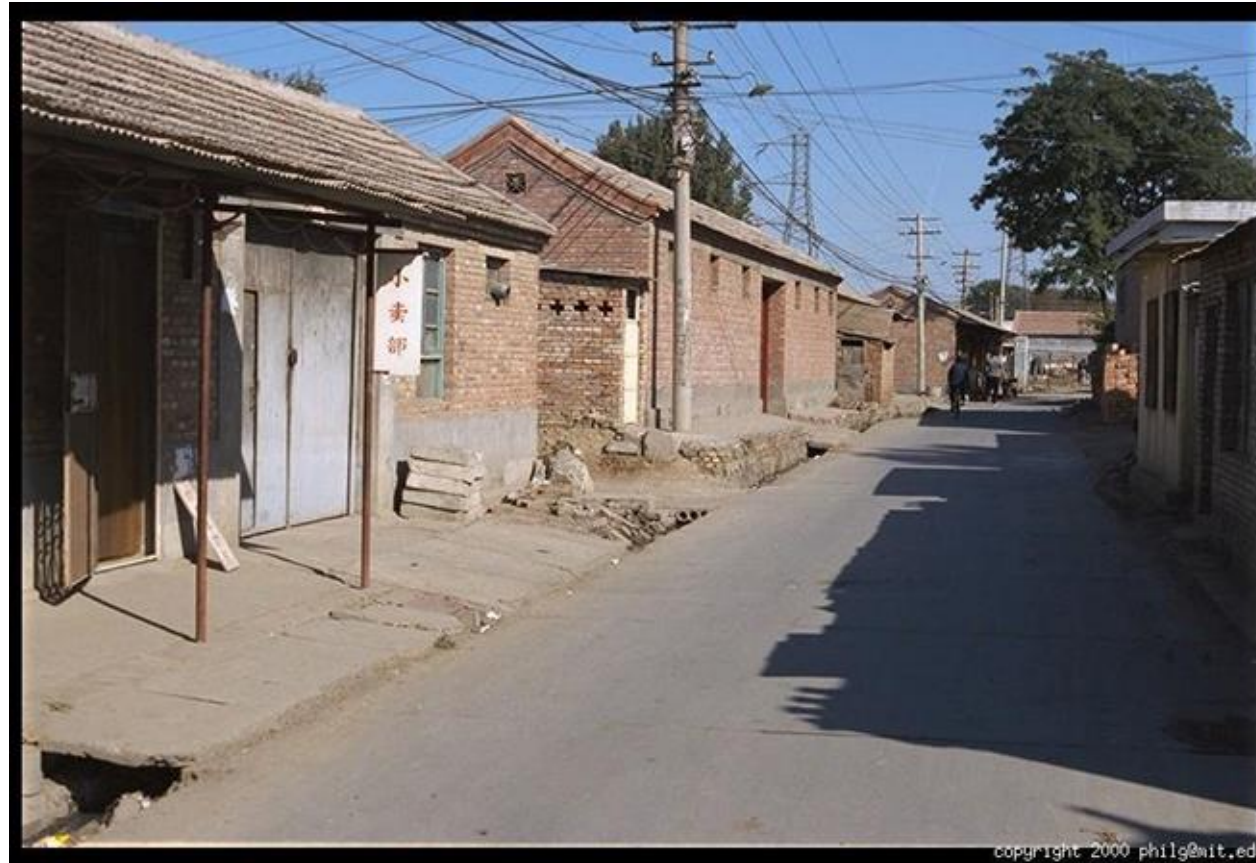


G = 1
(R=0,B=0)



B = 1
(R=0,G=0)

Most information in intensity



Original image

Most information in intensity



Only intensity shown – constant color

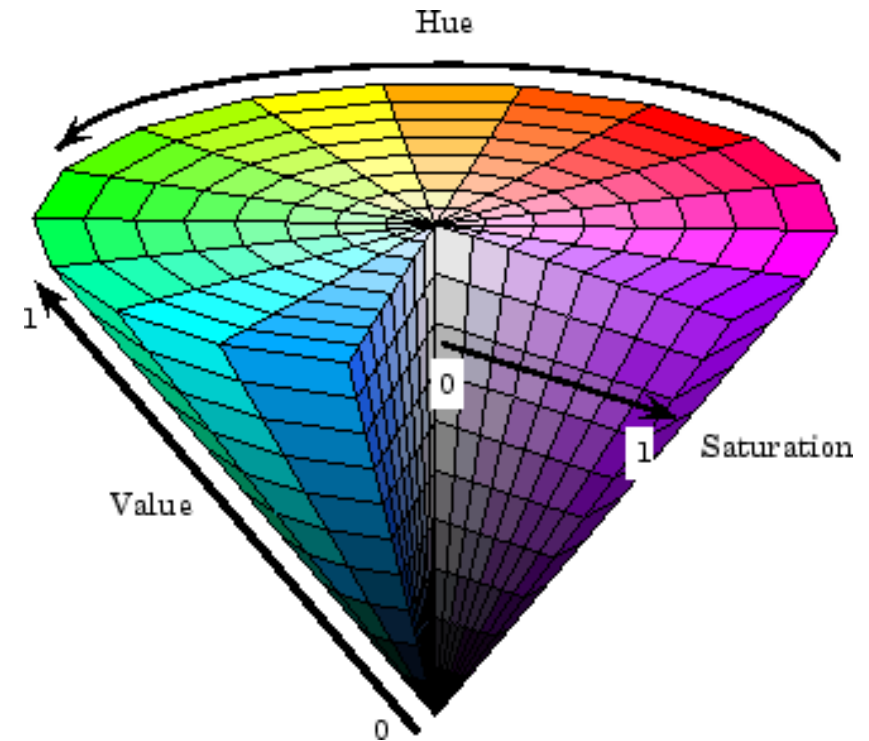
Most information in intensity



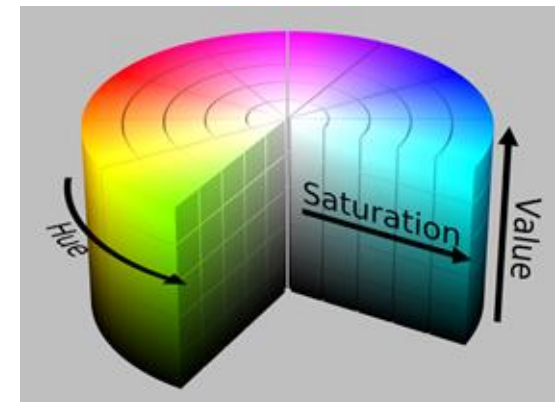
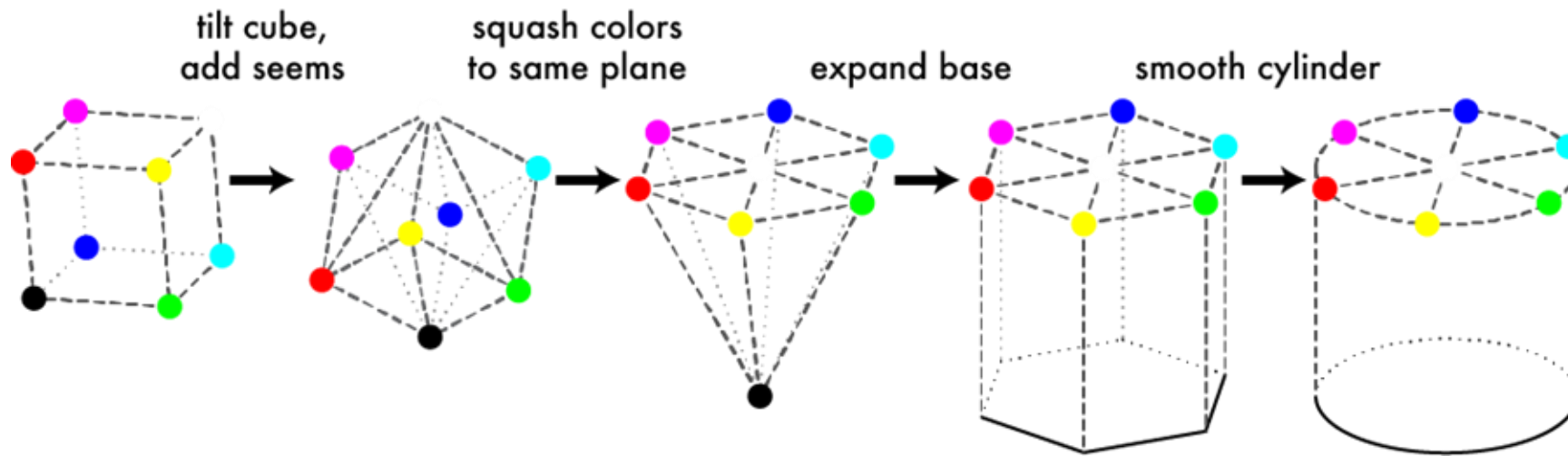
Only color shown – constant intensity

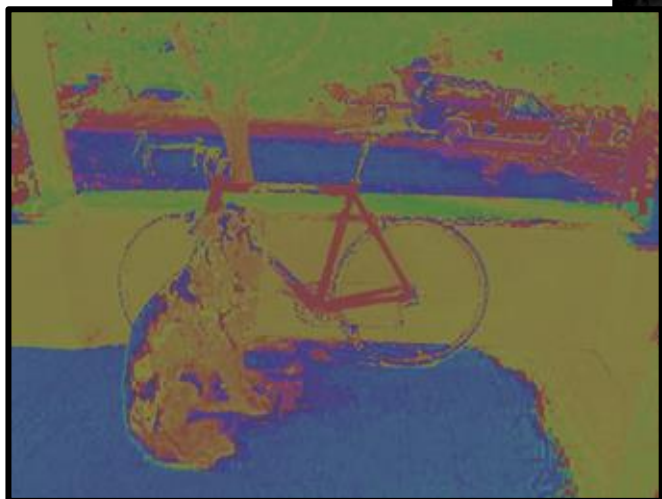
Color spaces: HSV

- Different model based on perception of light
- Intuitive color space
- **Hue**: what color
- **Saturation**: how much color
- **Value**: how bright
- Allows easy image transforms
 - Shift the hue
 - Increase saturation



Geometric RGB to HSV





Hue

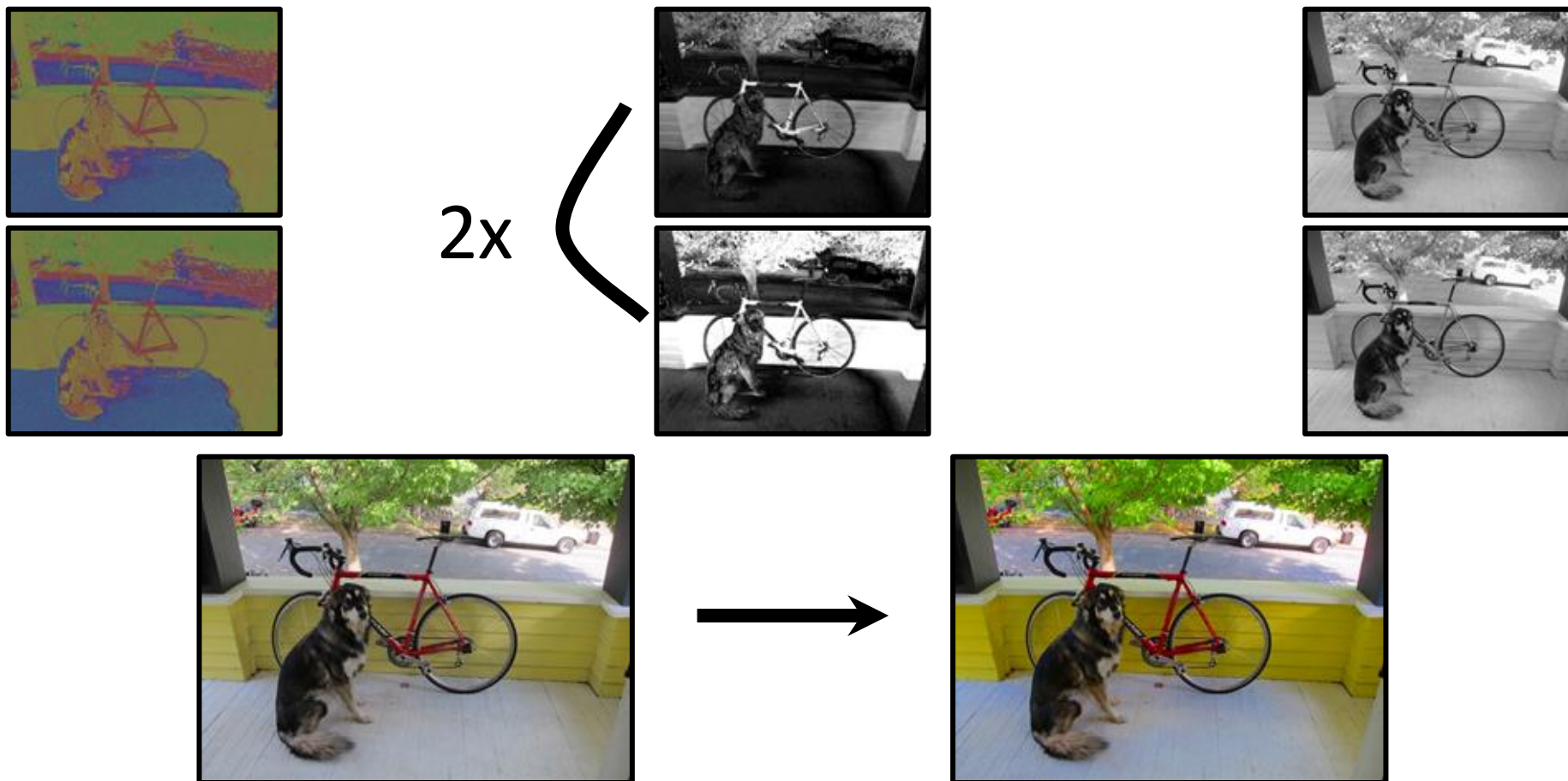


Saturation

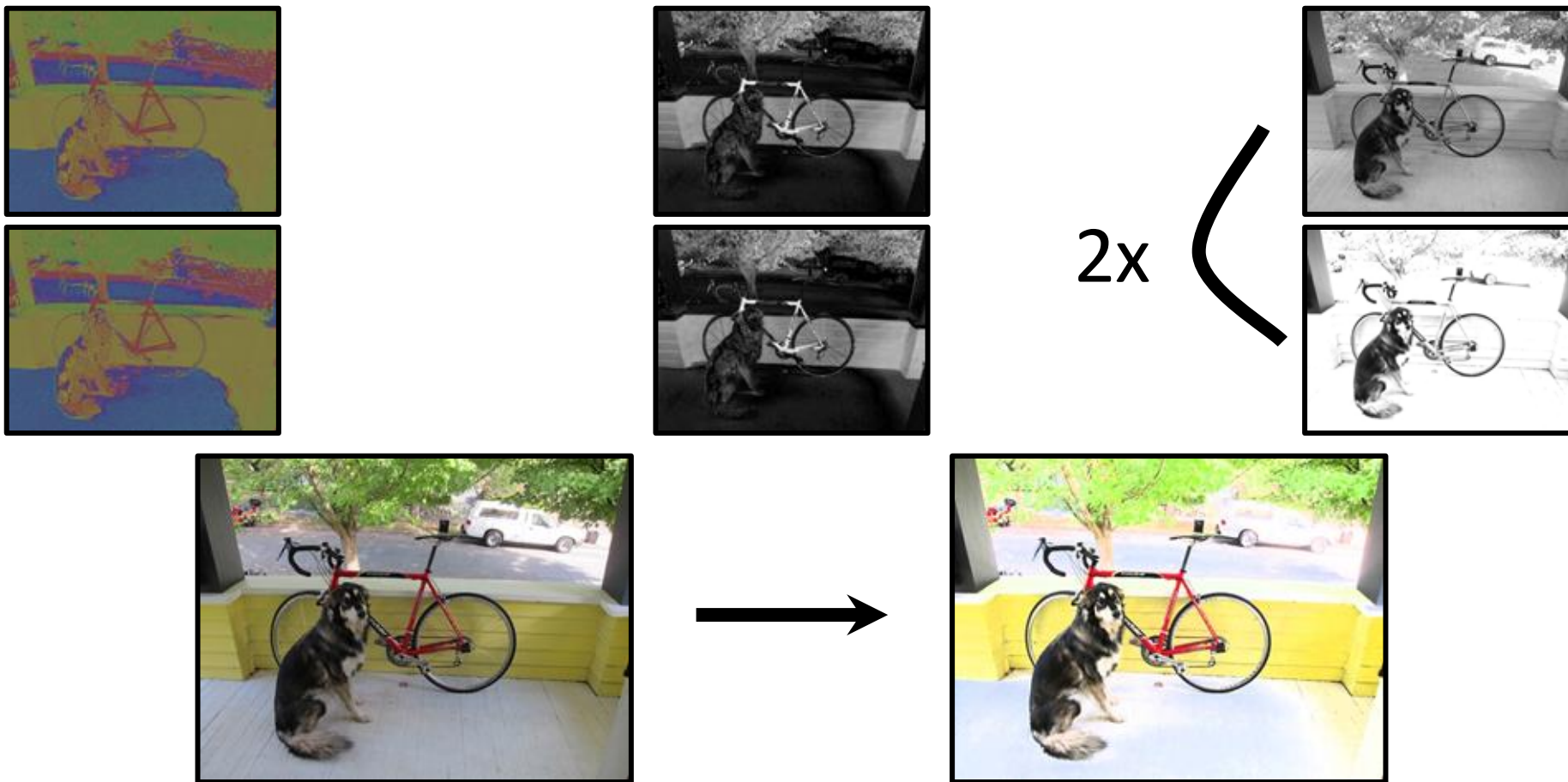


Value

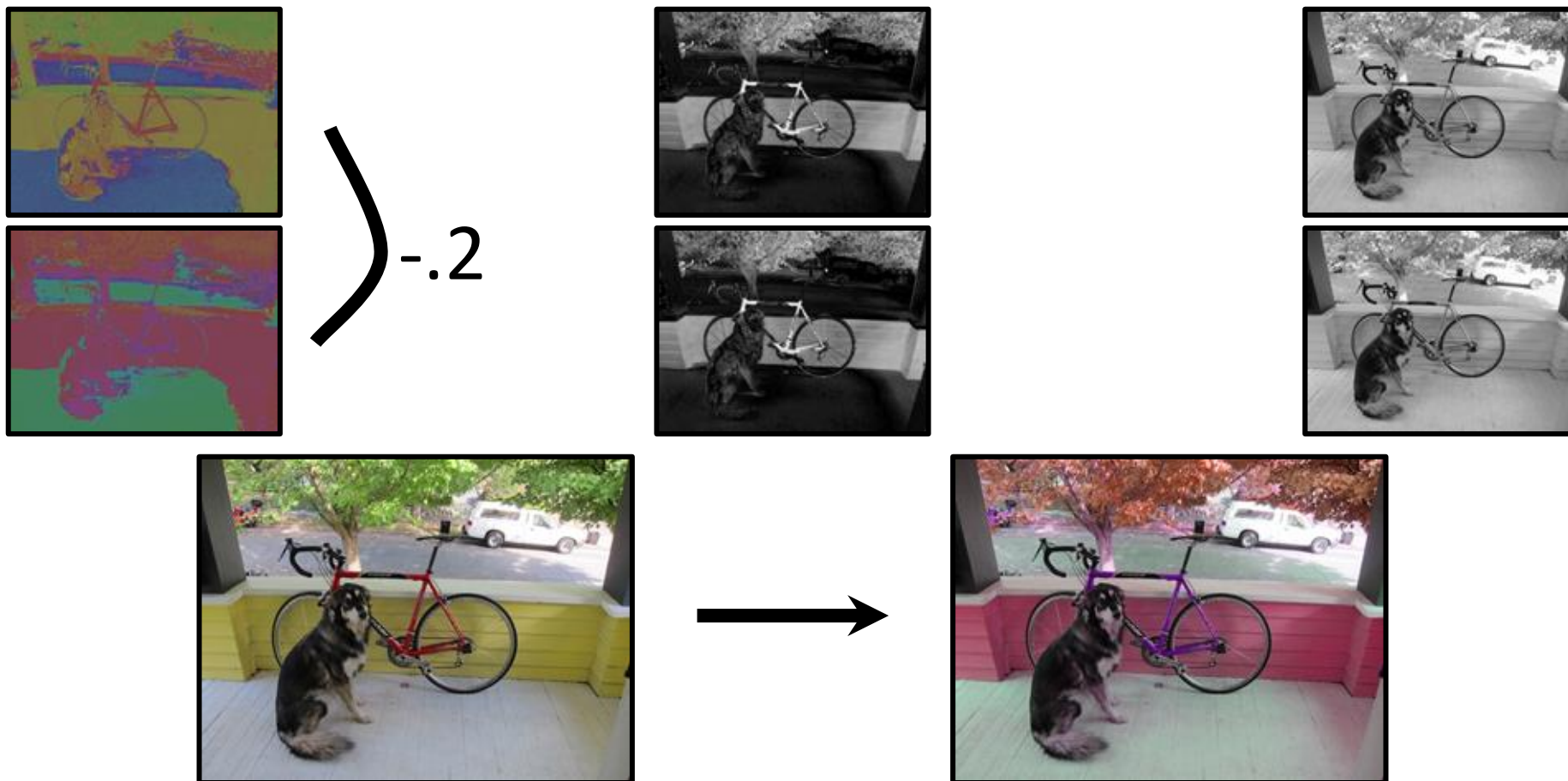
More saturation = intense colors



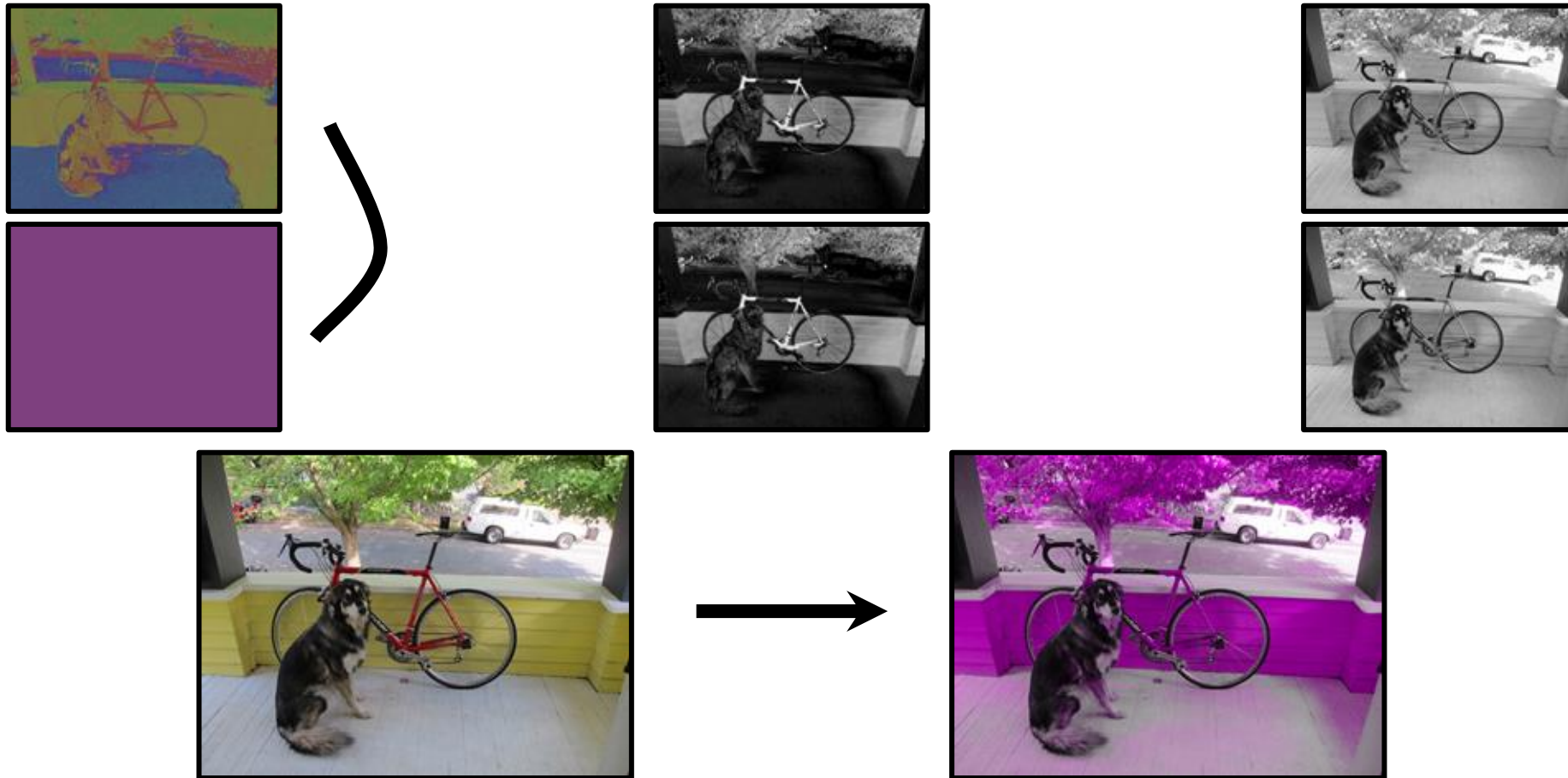
More value = lighter image



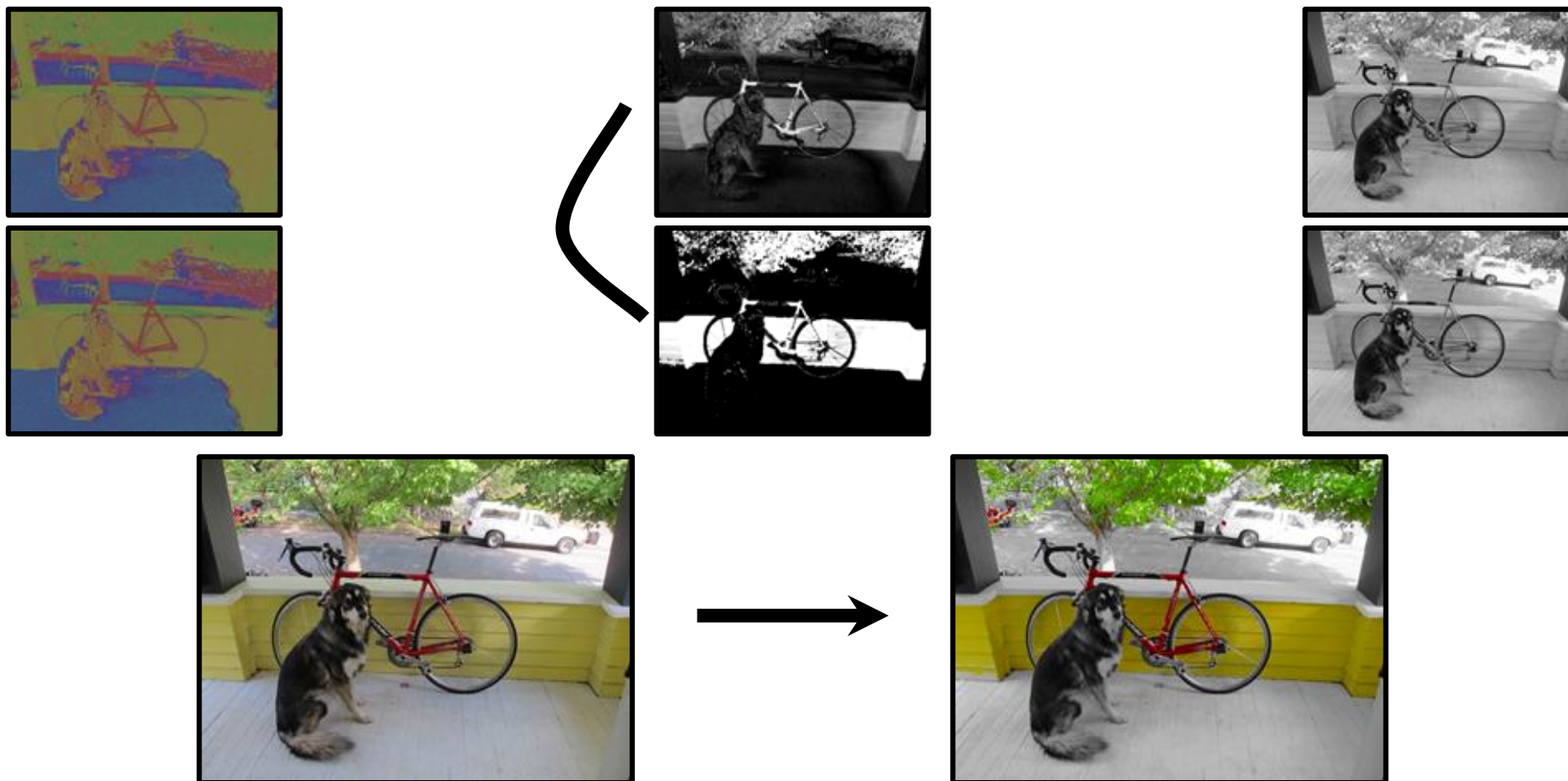
Shift hue = shift colors



Set hue to a given color



Increase and threshold saturation



2D Images

